대규모 트래픽 환경에서 데이터베이스 부하 분산을 위한 API 캐싱 전략 비교

수강신청 과목 조회 시스템을 중심으로

구현모
단국대학교
hmoo4198@dankook.ac.kr

A Comparative Study of API Caching Strategies for Database Load Distribution in High-Traffic Environments

Focusing on a Course Registration Course Lookup System

Koohyunmo Koo
Dankook University

Abstract

수강신청 시스템은 특정 시점에 대규모 사용자가 동시에 접속하는 대표적인 트래픽 집중형 서비스이며, 이 중 과목 조회 기능은 짧은 시간 동안 동일한 데이터에 대한 반복적인 읽기 요청이 집중되는 특성을 가진다. 이러한 조회 요청이 모두 데이터베이스로 직접 전달될 경우 커넥션 자원이 빠르게 소진되고, 실제 수강신청과 같은 핵심 쓰기 트랜잭션의 처리 성능까지 저하될 수 있다. 본 연구는 수강신청 과목 조회 API를 대상으로 캐시 미적용(No Cache), Caffeine 기반 로컬 캐시, Redis 기반 글로벌 캐시의 세 가지 전략을 구현하고, JMeter를 이용한 읽기 집중 부하 실험을 통해 응답 시간과 처리량을 비교하였다. 실험 결과, No Cache 환경은 평균 응답시간 2110.67ms, TPS 79.99를 기록하였고 p99는 10844.00ms까지 증가하였다. 반면 Caffeine은 평균 6.99ms, TPS 200.30으로 가장 우수한 성능을 보였으며, Redis는 평균 283.84ms, TPS 174.14로 No Cache 대비 유의미한 성능 향상을 보였다. 이를 통해 읽기 집중형 API에서 캐시 도입은 데이터베이스 부하를 완화하는 데 매우 효과적이며, 단일 인스턴스 환경에서는 Caffeine, 확장성과 캐시 공유가 필요한 환경에서는 Redis가 더 적합한 선택이 될 수 있음을 확인하였다.

1. 서론

대학 수강신청 시스템은 일정 시점에 매우 짧은 시간 동안 다수의 사용자가 동시 접속하는 구조를 가진다. 이 과정에서 학생들은 실제 수강신청 버튼을 누르기 전에 과목 목록을 반복적으로 조회하며, 이 조회 요청은 대부분 동일한 데이터에 집중된다. 문제는 이러한 읽기 요청이 애플리케이션과 데이터베이스에 누적되면서 상대적으로 더 중요한 쓰기 트랜잭션인 수강신청 처리의 안정성까지 악화시킨다는 점이다.

실제 서비스 운영 관점에서 읽기 요청은 전체 트래픽의 대부분을 차지하면서도 데이터 변경 빈도는 상대적으로 낮은 경우가 많다. 과목 조회 API 역시 특정 실험 구간 동안 내용이 거의 변하지 않으므로, 캐시를 적용했을 때 효과를 정량적으로 측정하기에 적합하다. 특히 동일 데이터에 대한 반복 요청이 집중되는 상황은 캐시 적용 전후의 차이를 명확하게 관찰할 수 있게 한다.

본 연구의 목적은 수강신청 과목 조회 API에 캐시를 적용했을 때 데이터베이스 부하 분산과 응답 성능 개선 효과가 얼마나 발생하는지를 비교하는 데 있다. 이를 위해 캐시를 전혀 적용하지 않은 환경(No Cache), 애플리케이션 로컬 메모리를 사용하는 Caffeine 환경, 외부 캐시 저장소를 사용하는 Redis 환경을 각각 구현하였다. 이후 JMeter를 사용해 동일한 조건의 읽기 집중 부하를 가하고, 평균 응답시간, 상위 백분위 응답시간, TPS, 에러율을 비교하여 각 전략의 장단점을 분석하였다.

2. 문제 정의 및 연구 목표

2.1 문제 정의

수강신청 시나리오에서 병목은 반드시 쓰기 요청에서만 발생하지 않는다. 오히려 실제 신청 시작 직전의 조회 폭주가 먼저 데이터베이스 자원을 소모하여 전체 시스템 안정성을 떨어뜨릴 수 있다. 과목 조회 API가 모든 요청마다 데이터베이스를 직접 조회하도록 설계되어 있다면, 동일한 과목 목록이 매우 짧은 시간 안에 수천 번 반복 조회되는 비효율이 발생한다.

이러한 구조는 다음과 같은 문제를 유발한다.

- 데이터베이스 커넥션 사용량 증가
- 조회 지연 시간 증가와 tail latency 확대

- 핵심 쓰기 트랜잭션의 응답성 저하 가능성
- 고부하 시 오류 응답 발생 가능성 증가

2.2 연구 목표

본 연구는 위 문제를 해결하기 위한 실용적 접근으로 캐시를 적용하고, 다음 두 가지 질문에 답하고자 한다.

1. 읽기 집중형 과목 조회 API에 캐시를 적용하면 성능이 실제로 얼마나 개선되는가?
2. 로컬 캐시(Caffeine)와 글로벌 캐시(Redis)는 어떤 환경에서 더 적합한가?

3. 시스템 구성 및 구현

3.1 도메인 구성

실험 대상 시스템은 수강신청 도메인을 단순화하여 `Course`, `Student`, `Enrollment` 세 개의 엔티티로 구성하였다. 이 중 본 연구의 실험 대상은 `Course` 목록 조회 API이며, 서버 기동 시 더미 데이터 생성 로직을 통해 과목 데이터 10,000건을 적재하였다. 과목 코드는 `CS00001`부터 `CS10000`까지 순차적으로 생성하였다.

3.2 캐시 전략 구현

모든 전략은 동일한 응답 DTO와 동일한 조회 대상 데이터를 사용하며, 차이는 캐시 계층의 유무와 위치에만 존재한다.

No Cache 방식은 `/api/v1/courses/no-cache` 엔드포인트 호출 시 매 요청마다 JPA Repository를 통해 전체 과목을 조회한다. 구현은 가장 단순하지만, 모든 요청이 직접 DB를 사용한다.

Caffeine 방식은 `/api/v1/courses/caffeine` 엔드포인트에 대해 애플리케이션 내부 메모리에 과목 목록을 저장한다. 캐시 TTL은 5분이며, 동일 키 요청은 DB 대신 로컬 메모리에서 즉시 반환된다. Caffeine은 시간 기반 만료와 크기 기반 제거를 함께 지원하는 고성능 인메모리 캐시 구현체로 알려져 있다 [2]. 단일 인스턴스 환경에서는 가장 짧은 응답 시간을 기대할 수 있다.

Redis 방식은 `/api/v1/courses/redis` 엔드포인트에 대해 Redis CacheManager를 사용하여 과목 목록을 외부 캐시에 저장한다. TTL은 동일하게 5분이며, 로컬 캐시보다 네트워크 및 직렬화 비용이 추가되지만 여러 인스턴스 간 캐시 공유가 가능하다. Redis는 cache-aside 방식과 TTL을 활용해 읽기 중심 워크로드를 처리하는 대표적인 외부 캐시 저장소이다 [3].

3.3 실험 환경

표 1는 본 연구의 실험 환경을 정리한 것이다.

Table 1: 실험 환경

구분	내용
언어 및 프레임워크	Java 17, Spring Boot, Spring Data JPA
데이터베이스	H2 In-Memory Database
캐시 기술	Caffeine, Redis 7
부하 도구	Apache JMeter
데이터 규모	과목 데이터 10,000건
실행 방식	단일 애플리케이션 인스턴스
Redis 실행 환경	Docker 컨테이너 기반 로컬 실행

4. 실험 방법

세 가지 전략은 모두 동일한 API 계약과 동일한 데이터셋을 사용하였으며, 성능 비교를 위해 JMeter로 동일한 부하 조건을 적용하였다. 부하 테스트 조건은 동시 사용자 300명, Ramp-Up 30초, 각 사용자당 20회 반복 요청이며, 전략별 총 요청 수는 6000건이다. JMeter는 테스트 플랜 기반의 HTTP 부하 생성과 지표 수집을 지원하므로, 웹 애플리케이션 성능 측정 도구로 널리 사용된다 [1].

실험 과정에서는 각 전략을 독립적으로 실행하여 캐시 상태가 서로 간섭하지 않도록 하였다. 응답 성능 평가는 평균 응답시간뿐 아니라, 사용자 경험에 더 직접적인 영향을 주는 p95 및 p99 지표와 TPS, 에러율을 함께 확인하였다. 이는 평균값만으로는 고부하 상황의 응답 특성을 충분히 설명할 수 없기 때문이다.

5. 실험 결과

표 2는 세 가지 전략의 측정 결과를 정리한 것이다.

Table 2: 캐시 전략별 성능 비교 결과

전략	평균(ms)	p95(ms)	p99(ms)	TPS	에러율(%)
No Cache	2110.67	6964.00	10844.00	79.99	0.07
Caffeine	6.99	13.00	32.01	200.30	0.00
Redis	283.84	528.00	616.01	174.14	0.00

No Cache 방식은 평균 응답시간이 2110.67ms로 가장 높았고, p95와 p99가 각각 6964.00ms, 10844.00ms까지 증가하였다. 이는 조회 요청이 모두 데이터베이스에 직접 도달하면서 고부하 환경에서 응답 지연이 급격히 누적되었음을 의미한다. 총 6000건 중 4건의 실패 요청이 발생했다는 점도 고부하 구간의 안정성 한계를 보여준다.

Caffeine 방식은 평균 응답시간 6.99ms, p95 13.00ms, p99 32.01ms로 가장 우수한 성능을 나타냈다. 이는 캐시 데이터가 애플리케이션 내부 메모리에 존재하여 데이터베이스 접근 비용과 네트워크 왕복 비용이 모두 제거되었기 때문이다. TPS 역시 200.30으로 가장 높게 나타나, 단일 인스턴스 기준 조회 성능 최적화에 가장 적합한 전략임을 확인할 수 있었다.

Redis 방식은 평균 응답시간 283.84ms, p95 528.00ms, p99 616.01ms를 기록하였다. 절대 성능은 Caffeine보다 낮지만, No Cache와 비교하면 응답시간과 처리량 모두에서

큰 개선 효과를 보였다. 이는 Redis가 DB 부하를 효과적으로 분산시키면서도 외부 캐시 서버와의 통신 오버헤드를 함께 가진다는 점을 잘 보여준다.

6. 결과 분석 및 논의

6.1 성능 관점의 해석

실험 결과에서 가장 먼저 확인할 수 있는 점은 캐시 적용 여부 자체가 성능에 지배적인 영향을 준다는 사실이다. No Cache 방식은 평균값도 높지만, p95와 p99가 비정상적으로 크게 나타났다. 이는 일부 요청이 매우 오랜 시간 지연되었다는 뜻이며, 실제 사용자 경험 관점에서는 평균 응답시간보다 더 치명적인 문제다.

반면 Caffeine은 거의 모든 요청이 매우 짧은 시간 안에 처리되었고, tail latency 또한 매우 낮게 유지되었다. 이는 단일 프로세스 내부에서 곧바로 데이터를 반환하는 로컬 캐시의 구조적 이점 때문이다. Redis는 로컬 메모리만큼 빠르지는 않지만, DB를 직접 조회하는 방식과 비교하면 응답시간과 안정성 측면에서 충분히 유의미한 개선을 보였다.

6.2 아키텍처 관점의 해석

Caffeine이 가장 빠르다는 결과만으로 모든 상황에서 최선이라고 결론 내릴 수는 없다. Caffeine은 각 애플리케이션 인스턴스가 개별 캐시를 가지므로, 다중 인스턴스 환경에서는 캐시 일관성 관리가 어려워질 수 있다. 반면 Redis는 외부 캐시 저장소를 공유하므로 여러 인스턴스가 동일한 캐시를 참조할 수 있어 수평 확장에 유리하다.

따라서 단일 서버 환경에서는 Caffeine이 가장 효율적일 수 있지만, 실제 서비스처럼 여러 애플리케이션 인스턴스가 운영되는 구조에서는 Redis가 더 현실적인 선택이 될 수 있다. 즉, 본 실험은 단순한 속도 비교를 넘어 캐시 전략 선택이 시스템 아키텍처와 밀접하게 연결되어 있음을 보여준다.

6.3 연구 질문에 대한 답변

본 연구의 첫 번째 질문인 "캐시 적용이 실제로 성능을 개선하는가"에 대해서는 명확히 그렇다고 답할 수 있다. Caffeine과 Redis 모두 No Cache 대비 응답시간과 TPS에서 큰 폭의 개선을 보였다.

두 번째 질문인 "로컬 캐시와 글로벌 캐시 중 무엇이 더 적합한가"에 대해서는 환경 의존적이라는 결론이 타당하다. 단일 인스턴스 환경에서는 Caffeine이 가장 우수하지만, 확장성과 캐시 공유가 중요한 운영 환경에서는 Redis가 더 실용적이다.

7. 한계점

본 연구에는 몇 가지 한계가 존재한다. 첫째, 데이터베이스로 MySQL 대신 H2를 사용하였기 때문에 실제 운영 환경과 동일한 절대 성능을 재현했다고 보기는 어렵다. 따라서 본 연구 결과는 캐시 전략 간 상대적 차이에 초점을 두고 해석해야 한다.

둘째, 실험은 단일 애플리케이션 인스턴스 환경에서 수행되었다. 실제 서비스 환경에서는 다중 인스턴스, 로드밸런서, 네트워크 지연, 커넥션 풀 설정, JVM 튜닝 등의 요소가 성능에 추가적인 영향을 미칠 수 있다.

셋째, 본 연구는 조회 API 중심 비교에 집중하였으므로, 수강신청과 같은 쓰기 요청이 발생할 때의 캐시 무효화 전략과 데이터 정합성 보장 문제를 정밀하게 다루지는 않았다. 향후 연구에서는 쓰기 트랜잭션과 연계된 캐시 일관성 전략을 포함한 비교가 필요하다.

8. 결론

본 연구에서는 수강신청 과목 조회 시스템을 대상으로 No Cache, Caffeine, Redis 세 가지 전략을 구현하고, 읽기 집중 부하 상황에서의 성능을 비교하였다. 실험 결과, 캐시 미적용 방식은 가장 높은 응답시간과 가장 낮은 처리량을 보였고, Caffeine은 가장 우수한 응답 성능과 가장 높은 TPS를 기록하였다. Redis는 로컬 캐시보다는 느렸지만, DB 직조회 방식 대비 매우 유의미한 성능 개선을 제공하였다.

결론적으로 읽기 집중형 API에서는 캐시 도입이 데이터베이스 병목 완화에 매우 효과적이며, 단일 인스턴스 환경에서는 Caffeine, 확장성과 캐시 공유가 중요한 환경에서는 Redis가 더 적합한 전략이 될 수 있다. 본 연구는 수강신청과 같은 트래픽 집중형 서비스에서 캐시 전략 선택의 실질적 기준을 제시했다는 점에서 의미가 있다. 이러한 결론은 웹 캐싱이 지연 시간과 서버 부하를 줄이는 데 효과적이라는 기존 연구의 흐름과도 일치한다 [4].

참고문헌

[1] Apache Software Foundation. Apache jmeter user's manual. https://jmeter.apache.org/usermanual/index.html, 2026. Accessed: 2026-03-27.

[2] Ben Manes. Eviction · ben-manes/caffeine wiki. https://github.com/ben-manes/caffeine/wiki/Eviction, 2025. Accessed: 2026-03-27.

[3] Redis. Caching — redis. https://redis.io/solutions/caching/, 2026. Accessed: 2026-03-27.

[4] Jia Wang. A survey of web caching schemes for the internet. *ACM SIGCOMM Computer Communication Review*, 29(5):36–46, 1999. doi: 10.1145/505696.505701.